



**ÉCOLE MAROCAINE DES
SCIENCES DE L'INGÉNIEUR**

Membre de **HONORIS UNITED UNIVERSITIES**

ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

EMSI Casablanca

projet python framework(django)

Système de Réservation Intelligente des Salles (SRIS)



Réalisé par :

Fatine Elbari & Salma Karim

Filière : 3ème année Ingénierie Informatique (3IIR)

Encadrant : Mme.Kassid Asmaa

Année universitaire : 2025 – 2026

Date : Mai 2026

Remerciements

Nous tenons à exprimer notre sincère gratitude envers toutes les personnes qui ont contribué à la réalisation de ce projet

Nos remerciements s'adressent en premier lieu à notre encadrant,

Mme.Kassid Asmaa, pour ses précieux conseils, sa disponibilité et son soutien tout au long de ce projet.

Nous remercions également le corps professoral de l'**EMSI Casablanca**, pour la qualité de l'encadrement pédagogique dispensé au cours de notre formation.

Enfin, nous exprimons notre reconnaissance envers nos familles et amis pour leur soutien moral indéfectible.

Résumé

Ce rapport présente le développement du Système de Réservation Intelligente des Salles (SRIS). Face à la complexité croissante de la gestion des salles dans un environnement universitaire, ce système propose une solution numérique permettant de consulter, réserver et gérer les espaces de l'établissement de manière simple et fiable. L'application, développée avec le framework Django (Python) et une base de données SQLite, offre des fonctionnalités de réservation en ligne, de détection automatique des conflits, de visualisation calendaire et de tableau de bord d'administration.

Mots-clés : *Réservation de salles, Django, REST API, Python, SQLite, gestion universitaire, interface web.*

Abstract

This report presents the development of the Intelligent Room Reservation System (SRIS). Faced with the growing complexity of room management in a university environment, this system provides a digital solution enabling users to browse, book, and manage institutional spaces in a simple and reliable way. The application, built with the Django framework (Python) and an SQLite database, offers online booking, automatic conflict detection, calendar visualization, and a comprehensive administration dashboard.

Keywords: *Room booking, Django, REST API, Python, SQLite, university management, web interface.*

Table des matières

Remerciements.....	i
Résumé.....	ii
Abstract.....	iii
Liste des figures.....	vi
Liste des tableaux.....	vii
Liste des abréviations.....	viii
Introduction générale.....	1
1 Présentation de l'école et du contexte.....	2
1.1 Présentation de l'EMSI Casablanca.....	2
1.2 Contexte général du projet.....	2
1.3 Parc immobilier de l'EMSI.....	3
2 Présentation du projet.....	4
2.1 Problématique.....	4
2.2 Objectifs du projet.....	4
2.2.1 Objectif général.....	4
2.2.2 Objectifs spécifiques.....	4
2.3 Cahier des charges.....	5
3 Analyse et conception.....	7
3.1 Analyse des besoins.....	7
3.1.1 Besoins fonctionnels.....	7
3.1.2 Besoins non fonctionnels.....	7
3.2 Diagramme de cas d'utilisation.....	7
3.3 Diagramme de classes.....	8
3.4 Diagrammes de séquence.....	10
3.4.1 Séquence : Authentification.....	10
3.4.2 Séquence : Création d'une réservation.....	11
3.5 Architecture du système.....	12
3.5.1 Architecture MVT Django.....	12
3.5.2 Modèle de données (SQLite).....	12
4 Réalisation et développement.....	13
4.1 Environnement de développement.....	13
4.2 Structure du projet Django.....	13

4.3 API REST avec Django REST Framework.....	14
4.3.1 Exemple de réponse API – Booking List.....	14
4.3.2 Formulaire de création de réservation via l'API.....	16
4.4 Interfaces utilisateur – Frontend.....	16
4.4.1 Tableau de bord (Dashboard).....	16
4.4.2 Cartographie du campus.....	17
4.4.3 Liste des salles (interface Étudiant / Enseignant).....	18
4.4.4 Salles les plus sollicitées.....	20
4.4.5 Gestion des réservations (Mes demandes).....	21
4.4.6 Calendrier des réservations.....	22
4.5 interfaces admin –frontend.....	23
4.5.1 tableau de bord(Dashboard) admin	23
4.5.2 Interface de modération (Vue Administration).....	24
4.5.3 Vue capacitaire et statistiques (Admin Center EMSI)	25
4.6 Détection des conflits de réservation.....	26
5 Résultats, tests et validation.....	28
5.1 Résultats obtenus.....	28
5.2 Tests réalisés.....	29
5.2.1 Tests fonctionnels.....	29
5.2.2 Tests de performance.....	29
5.3 Analyse critique.....	29
5.3.1 Points forts.....	29
5.3.2 Limites et axes d'amélioration.....	30
Conclusion et perspectives.....	31
A Cahier des charges complet.....	32
B Extraits de code source.....	33

Liste des figures

Figure 2.1 – Diagramme de Gantt – Planification du projet SRIS	6
Figure 3.1 – Diagramme de cas d'utilisation – SRIS	8
Figure 3.2 – Diagramme de classes détaillé – SRIS	9
Figure 3.3 – Diagramme de séquence – Authentification	10
Figure 3.4 – Diagramme de séquence – Création d'une réservation	11
Figure 3.5 – Architecture MVT du projet Django SRIS	12
Figure 4.1 – Interface DRF – Booking List (GET /api/bookings/)	15
Figure 4.2 – Formulaire DRF – Création de réservation (POST /api/bookings/)	16
Figure 4.3 – Interface – Tableau de bord (Dashboard)	17
Figure 4.4 – Cartographie du campus – EMSI (5 étages)	18
Figure 4.5 – Interface – Liste des salles EMSI	19
Figure 4.6 – Interface – Répartition des salles par étage	20
Figure 4.7 – Interface – Salles les plus sollicitées	21
Figure 4.8 – Interface – Mes demandes de réservation	22
Figure 4.9 – Interface – Calendrier des réservations (vue semaine)	23

Liste des tableaux

Tableau 1.1 – Informations générales sur l'EMSI Casablanca	2
Tableau 1.2 – Parc immobilier de l'EMSI par étage	3
Tableau 2.1 – Cahier des charges – Spécifications	5
Tableau 3.1 – Modèle de données Booking (SQLite)	12
Tableau 4.1 – Environnement de développement	13
Tableau 4.2 – Endpoints de l'API REST	14
Tableau 5.1 – Résultats obtenus par fonctionnalité	25
Tableau 5.2 – Tests fonctionnels réalisés	26
Tableau 5.3 – Tests de performance	26

Liste des abréviations

Abréviation	Signification
SRIS	Système de Réservation Intelligente des Salles
EMSI	École Marocaine des Sciences de l'Ingénieur
API	Application Programming Interface
REST	Representational State Transfer
MVT	Model – View – Template
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
UML	Unified Modeling Language
CRUD	Create, Read, Update, Delete
JSON	JavaScript Object Notation
JWT	JSON Web Token
HTTP	HyperText Transfer Protocol
HTML	HyperText Markup Language
CSS	Cascading Style Sheets

Introduction générale

La gestion des infrastructures universitaires représente un défi permanent pour les établissements d'enseignement supérieur. L'attribution des salles de cours, de laboratoires et d'espaces de conférence implique une coordination complexe entre enseignants, étudiants et services administratifs. Les méthodes traditionnelles, souvent manuelles, génèrent fréquemment des conflits de planning, des doubles réservations et une utilisation sous-optimale des espaces disponibles.

C'est dans ce contexte que s'inscrit notre Projet : le développement du Système de Réservation Intelligente des Salles (SRIS) pour l'EMSI Casablanca. Ce projet vise à mettre en place une application web moderne, accessible depuis tout navigateur, permettant à l'ensemble des acteurs de l'établissement de gérer les réservations de manière efficace, transparente et sécurisée.

Organisation du rapport : Ce document est structuré en cinq chapitres. Le premier présente l'école et le contexte du projet. Le second détaille la problématique et le cahier des charges. Le troisième expose la phase d'analyse et de conception. Le quatrième décrit la réalisation technique. Enfin, le cinquième chapitre présente les résultats, les tests et la validation du système.

CHAPITRE 1

Présentation de l'école et du contexte

1.1 Présentation de l'EMSI Casablanca

L'École Marocaine des Sciences de l'Ingénieur (EMSI) est un établissement d'enseignement supérieur privé marocain, membre du groupe Honoris United Universities. Fondée en 1986, elle forme des ingénieurs et des cadres dans les domaines de l'informatique, des télécommunications, du génie civil et des sciences de gestion.

Critère	Informations
Secteur d'activité	Enseignement supérieur privé – Ingénierie
Filiales	Plusieurs campus au Maroc (Casablanca, Rabat, Marrakech, Fès...)
Accréditation	Reconnue par l'État marocain ; membre du réseau Honoris
Effectif	Plus de 10 000 étudiants à l'échelle nationale

1.2 Contexte général du projet

L'EMSI Casablanca dispose de nombreuses salles réparties sur plusieurs étages : salles de cours (SC), laboratoires informatiques (LI) et salles de conférence (CONF). Avec l'augmentation des effectifs, la gestion manuelle de ces espaces est devenue source de conflits récurrents et d'une allocation inefficace des ressources.

La direction a exprimé le besoin d'un outil numérique centralisé permettant :

- De consulter en temps réel la disponibilité des salles
- De soumettre et suivre des demandes de réservation
- D'automatiser la détection des conflits d'horaires
- De générer des rapports d'utilisation des espaces

1.3 Parc immobilier de l'EMSI

Étage	Salles de cours (SC)	Laboratoires (LI)	Salle conférence	Total
Étage 1	SC101–SC108 (8 salles)	LI101, LI102	–	10
Étage 2	SC201–SC208 (8 salles)	LI201, LI202	CONF201	11
Étage 3	SC301–SC308 (8 salles)	LI301, LI302	–	10
Étage 4	SC401–SC408 (8 salles)	LI401, LI402	CONF401	11
Étage 5	SC501–SC508 (8 salles)	LI501, LI502	–	10
TOTAL	40 salles	10 laboratoires	2 salles	52

CHAPITRE 2

Présentation du projet

2.1 Problématique

La gestion actuelle des salles à l'EMSI Casablanca repose sur des méthodes manuelles ou semi-numériques (tableurs, emails) qui présentent plusieurs limitations majeures : absence de vue centralisée des disponibilités, risque de double réservation, délais de traitement des demandes, et manque de traçabilité des attributions. Ces lacunes justifient le développement d'un système automatisé et sécurisé.

2.2 Objectifs du projet

2.2.1 Objectif général

Développer une application web complète permettant la gestion intelligente des réservations de salles à l'EMSI Casablanca, en automatisant la détection des conflits, la validation des demandes et la visualisation des créneaux disponibles.

2.2.2 Objectifs spécifiques

- Permettre aux étudiants et enseignants de consulter les salles disponibles en temps réel
- Implémenter un système de réservation en ligne avec workflow de validation
- Détecter automatiquement les conflits d'horaires et rejeter les demandes incompatibles
- Offrir un tableau de bord d'administration avec indicateurs clés
- Exposer une API REST complète pour une intégration future
- Garantir la sécurité des accès via RBAC (4 rôles distincts)

2.3 Cahier des charges

Catégorie	Spécification
Acteurs	Étudiant, Enseignant, Administration, Administrateur Système
Fonctionnalités clés	Réservation, Consultation, Validation, Annulation, Rapports
Technologies	Django 4.x, DRF, SQLite, Bootstrap 5, FullCalendar.js
Sécurité	Authentification RBAC, Protection CSRF, Validation serveur
Performance	Temps de réponse < 2s pour toutes les opérations CRUD
Disponibilité	Interface web, accessible depuis tout navigateur moderne

CHAPITRE 3

Analyse et conception

3.1 Analyse des besoins

3.1.1 2Besoins fonctionnels

- Authentification différenciée par rôle (Étudiant, Enseignant, Administration, Admin Sys)
- Consultation des salles avec filtres (étage, type, capacité)
- Création, modification et annulation de réservations
- Validation/refus des réservations par l'administration
- Détection automatique des conflits d'horaires
- Calendrier visuel des réservations
- Tableau de bord avec statistiques en temps réel
- API REST pour interactions programmatiques

3.1.2 Besoins non fonctionnels

- Sécurité : protection CSRF, hachage des mots de passe, permissions RBAC
- Performance : temps de réponse < 1s pour les opérations courantes
- Maintenabilité : architecture MVT claire, code documenté
- Portabilité : interface responsive Bootstrap 5
- Fiabilité : validation des données côté serveur, contraintes DB

3.2 Diagramme de cas d'utilisation

Le diagramme ci-dessous présente l'ensemble des cas d'utilisation du système SRIS, organisés par acteur. Quatre acteurs principaux interagissent avec le système : Étudiant, Enseignant, Administration et Administrateur Système.

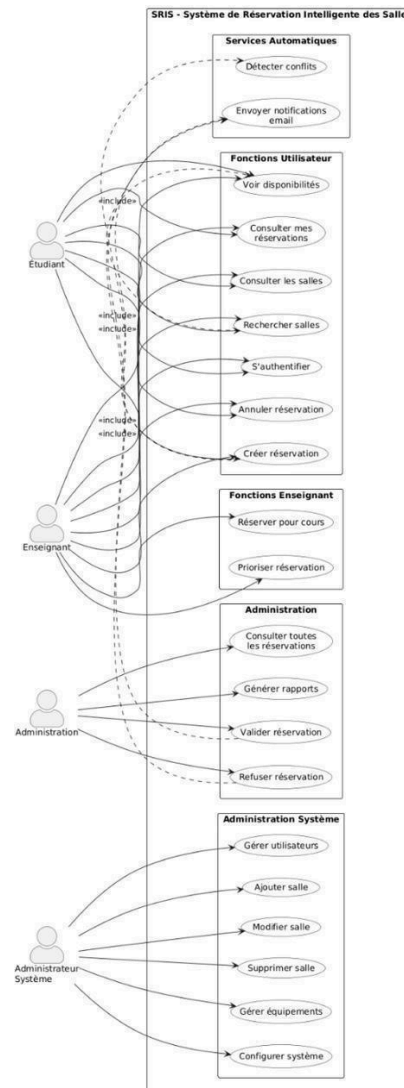


Figure 3.1 – Diagramme de cas d'utilisation – SRIS

3.3 Diagramme de classes

Le diagramme de classes détaillé modélise l'architecture orientée objet du système SRIS. Les entités principales sont : Utilisateur, Réservation, Salle, Créneau, Équipement et Rapport.

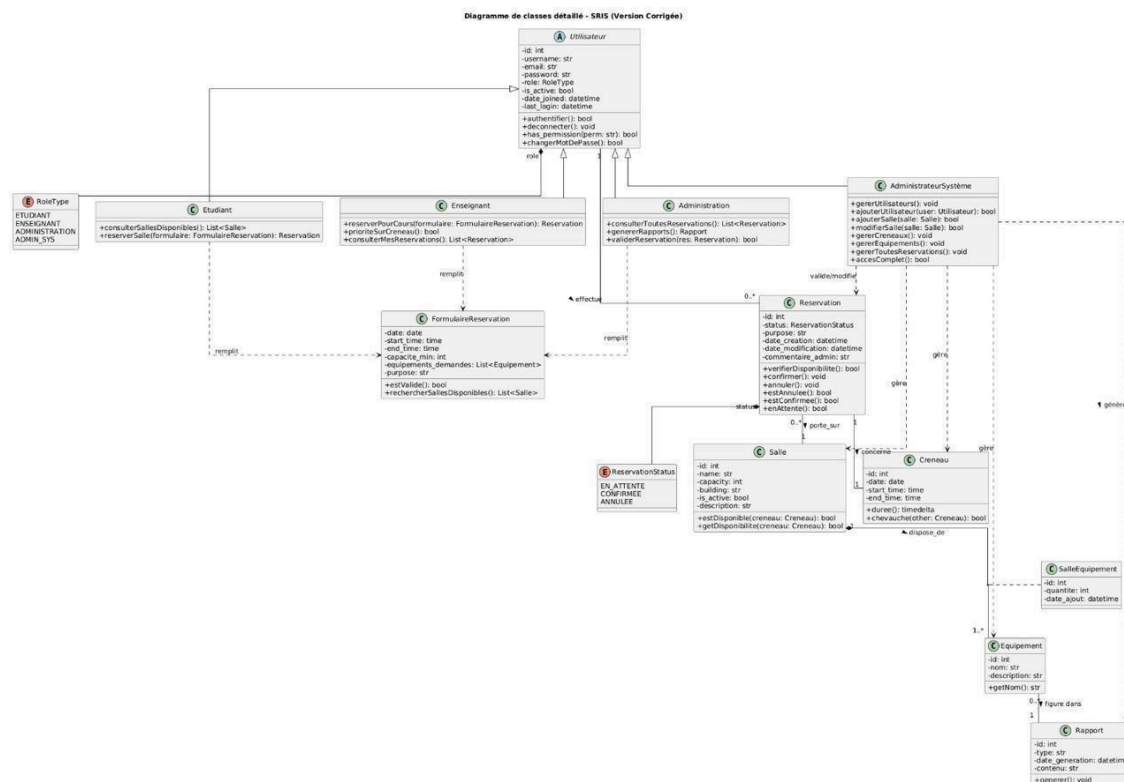


Figure 3.2 – Diagramme de classes détaillé – SRIS (Version Corrigée)

3.4 Diagrammes de séquence

3.4.1 Séquence : Authentification

Ce diagramme illustre le flux d'authentification d'un utilisateur. Le système vérifie les identifiants, récupère le rôle associé (ETUDIANT / ENSEIGNANT / ADMINISTRATION / ADMIN_SYS) et redirige vers le dashboard adapté.

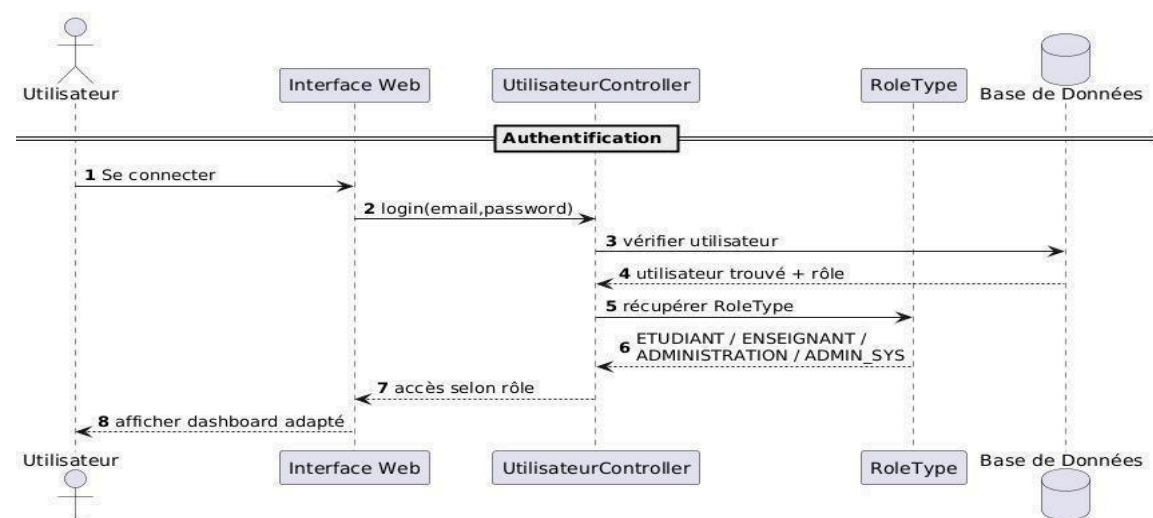


Figure 3.3 – Diagramme de séquence – Authentification

3.4.2 Séquence : Création d'une réservation

Ce diagramme détaille le processus complet de création d'une réservation, de la saisie du formulaire jusqu'à la confirmation par email, en passant par la vérification de disponibilité et la détection des conflits.

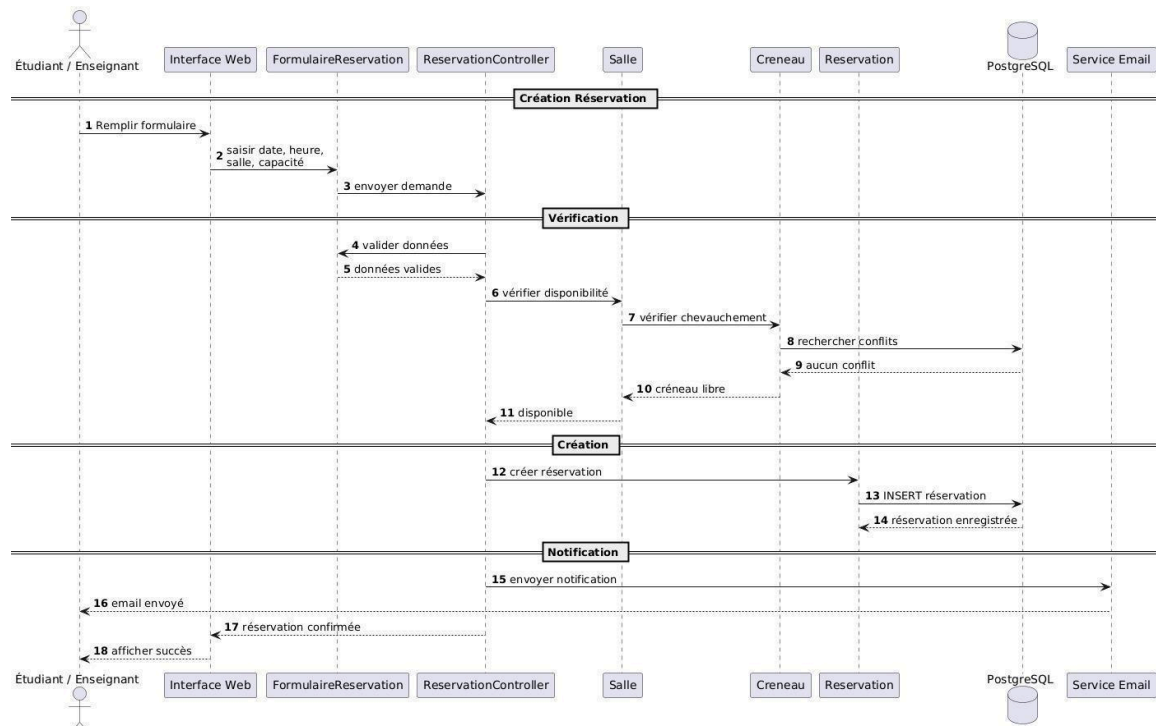


Figure 3.4 – Diagramme de séquence – Création d'une réservation

3.5 Architecture du système

3.5.1 Architecture MVT Django

Architecture MVT – Django SRIS

- Model → Entités (User, Room, Booking...) – accès
- ORM SQLite View → Logique métier + traitement des requêtes
- HTTP Template → Interface HTML (Bootstrap 5 + FullCalendar.js)
- DRF → Sérialisation REST (API JSON) Forms
- Formulaires avec validation CSRF Admin
- Interface administration Django

L'architecture MVT de Django sépare strictement les couches de présentation, de logique métier et d'accès aux données. Cette organisation garantit la maintenabilité du code et facilite les tests unitaires.

3.5.2 Modèle de données (SQLite)

Champ	Type	Description
user	ForeignKey(User)	Utilisateur demandeur
room	ForeignKey(Room)	Salle réservée
start_time	TimeField	Heure de début
end_time	TimeField	Heure de fin
status	CharField	EN_ATTENTE / CONFIRMEE / REFUSEE / ANNULEE
activity_type	CharField	COURS / TP / REUNION / CLUB / ATELIER / EVENEMENT
attendees_count	PositiveIntegerField	Nombre de participants
purpose	CharField	Motif de la réservation
admin_message	TextField	Message de l'administration

CHAPITRE 4

Réalisation et développement

4.1 Environnement de développement

Technologie/Outil	Version	Rôle
Python	3.11	Langage backend principal
Django	4.2	Framework web MVT
Django REST Framework	3.14	API REST
SQLite	3.x	Base de données
Bootstrap	5.3	Framework CSS frontend
FullCalendar.js	6.x	Calendrier interactif
VS Code	–	IDE de développement
Git	–	Contrôle de version
Postman	–	Tests API

4.2 Structure du projet Django

Le projet est organisé selon la structure Django standard, avec une séparation claire entre les applications métier, les templates et les ressources statiques.

```
RESERVATION-EMSI/
├── bookings/           (App Django - Réservations)
├── rooms/              (App Django - Salles)
├── users/              (App Django - Utilisateurs)
├── sris/               (App Django - Système de gestion)
├── core/               (Configuration principale)
├── templates/          (Templates HTML)
│   ├── bookings/
│   ├── registration/
│   ├── rooms/
│   └── sris/
├── static/             (Ressources statiques)
│   └── images/
└── Fichiers racine (manage.py, requirements.txt, etc.)
```

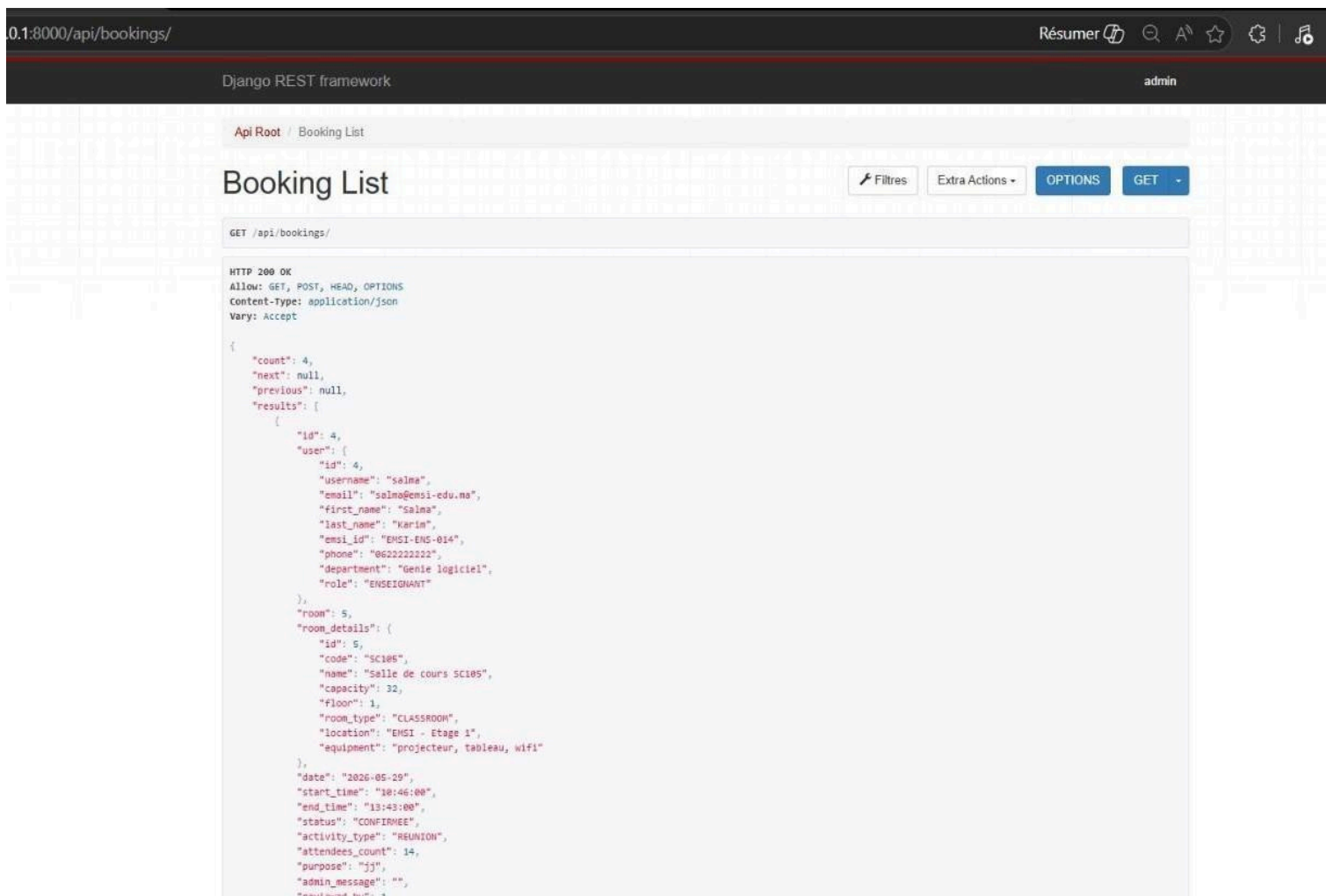
4.3 API REST avec Django REST Framework

Le système expose une API REST complète permettant des opérations CRUD sur les ressources principales. Chaque endpoint est sécurisé par authentification et permissions basées sur les rôles.

Endpoint	Méthode	Description	Rôle requis
GET /api/bookings/	GET	Liste des réservations	Authentifié
POST /api/bookings/	POST	Créer une réservation	Authentifié
GET /api/bookings/{id}/	GET	Détail réservation	Propriétaire/Admin
PUT /api/bookings/{id}/	PUT	Modifier réservation	Propriétaire/Admin
DELETE /api/bookings/{id}/	DELETE	Annuler réservation	Propriétaire/Admin
GET /api/rooms/	GET	Liste des salles	Authentifié
POST /api/rooms/	POST	Créer une salle	Admin Sys
GET /api/users/	GET	Liste des utilisateurs	Admin Sys

4.3.1 Exemple de réponse API – Booking List

L'interface Django REST Framework (DRF) offre une visualisation navigable de l'API. La capture suivante montre la liste des réservations avec les détails complets en format JSON.



4.3.2 Formulaire de création de réservation via l'API

L'interface DRF fournit également un formulaire HTML interactif pour tester la création de réservations directement depuis le navigateur.

The screenshot displays the Django REST framework interface. At the top, the header shows 'Django REST framework' on the left and 'admin' on the right. The main content area is divided into two sections. The top section shows a JSON response from an API call, with fields like 'status', 'activity_type', 'attendees_count', 'purpose', 'admin_message', 'reviewed_by', 'reviewed_by_name', 'reviewed_at', 'conflicts', and 'created_at'. The bottom section is an HTML form for creating a reservation. It includes fields for 'Room' (a dropdown menu), 'Date' (a date picker), 'Start time' and 'End time' (time pickers), 'Status' (a dropdown menu), 'Type d'activité' (a dropdown menu), 'Nombre de participants' (a text input), 'Purpose' (a text input), and 'Message de l'administration' (a text area). A 'POST' button is located at the bottom right of the form.

```
{
  "status": "CONFIRMEE",
  "activity_type": "COURS",
  "attendees_count": 28,
  "purpose": "Cours de genie logiciel",
  "admin_message": "",
  "reviewed_by": null,
  "reviewed_by_name": "",
  "reviewed_at": null,
  "conflicts": [],
  "created_at": "2026-05-14T23:40:16.313295+01:00"
}
```

Room: LI101

Date: jj/mm/aaaa

Start time: --:--

End time: --:--

Status: En attente

Type d'activité: Cours

Nombre de participants:

Purpose:

Message de l'administration:

POST

4.4 Interfaces utilisateur – Frontend

4.4.1 Tableau de bord (Dashboard)

Le tableau de bord constitue la page d'accueil principale du système. Il affiche en temps réel les indicateurs clés : nombre total de réservations, réservations du jour, salles actives, demandes en attente, ainsi qu'un résumé du parc EMSI et une cartographie du campus.

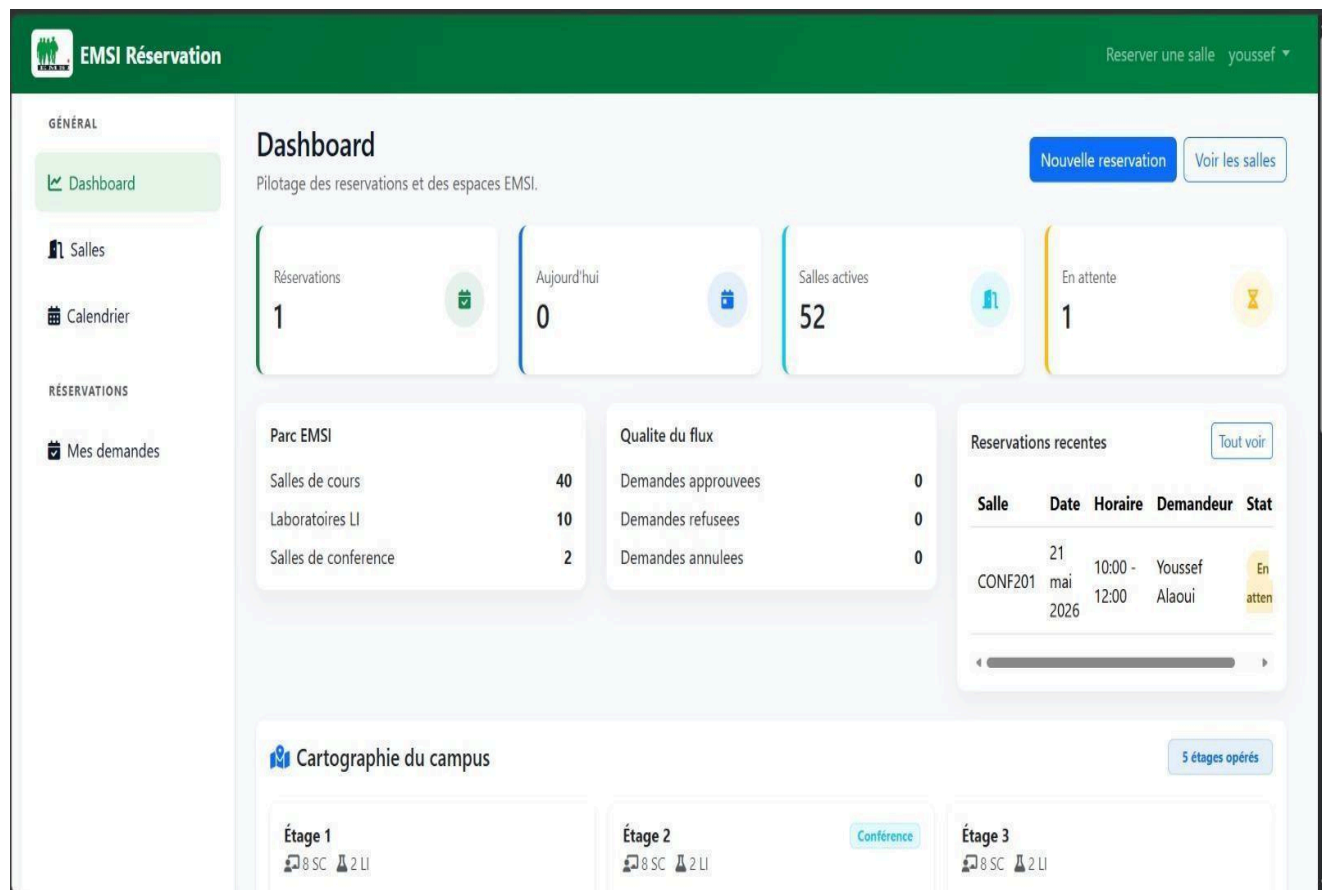


Figure 4.3 – Interface – Tableau de bord (Dashboard)

4.4.2 Cartographie du campus

La section Cartographie du campus offre une vue synthétique de l'organisation des salles par étage. Chaque étage liste ses salles de cours, ses laboratoires et ses éventuelles salles de conférence.

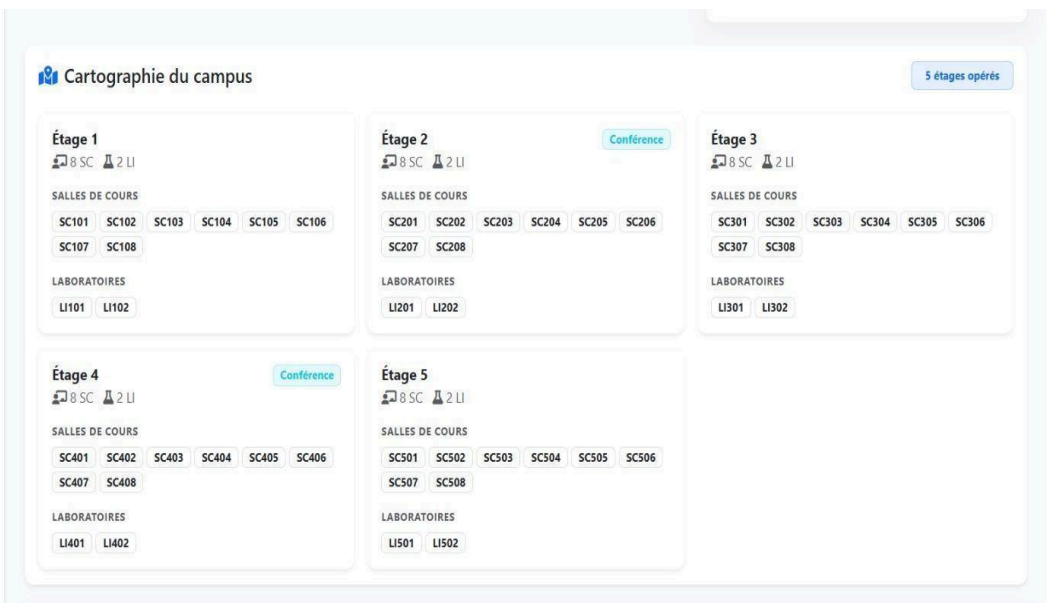
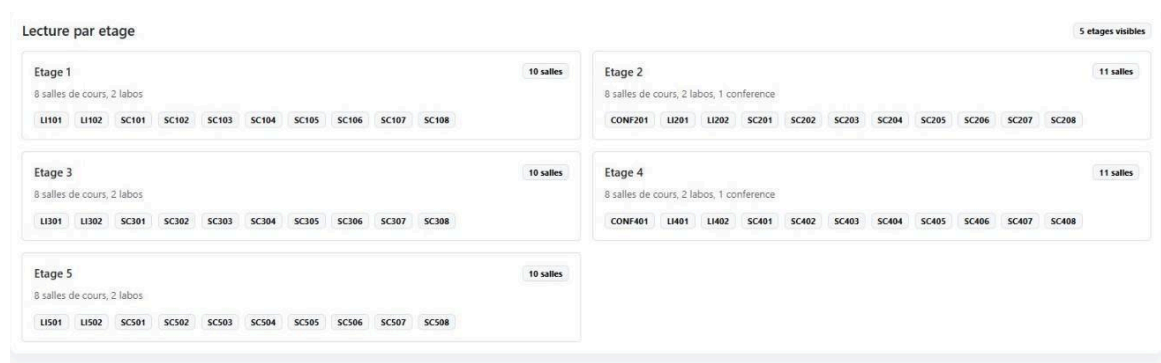
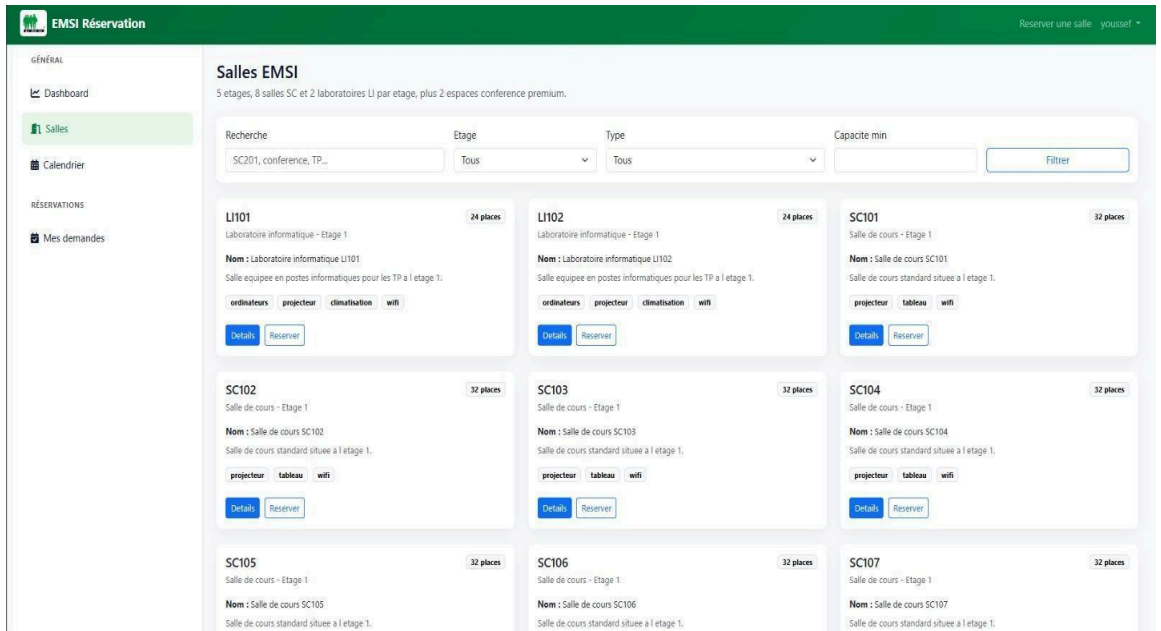


Figure 4.4 – Cartographie du campus – EMSI (5 étages)



4.4.3 Liste des salles

La page Salles EMSI présente l'ensemble des espaces disponibles avec des filtres avancés par étage, type et capacité minimale. Chaque salle dispose d'un bouton Détails et d'un bouton Réserver.



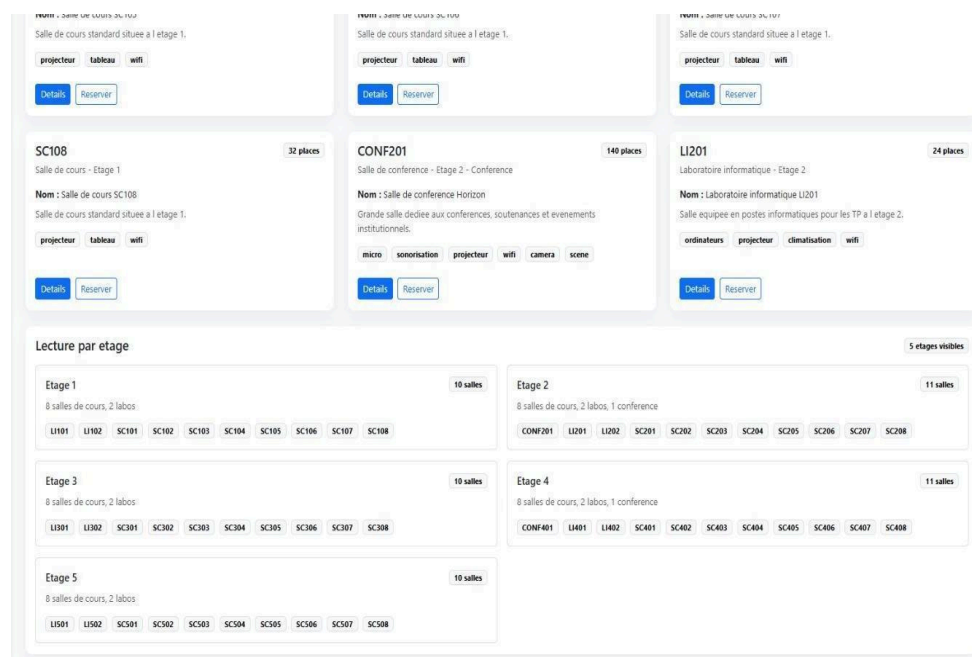


Figure 4.7 – Interface – Répartition des salles par étage

4.4.4 Salles les plus sollicitées

Le dashboard affiche un widget des salles les plus demandées, utile pour l'administration dans l'optimisation de l'allocation des espaces.

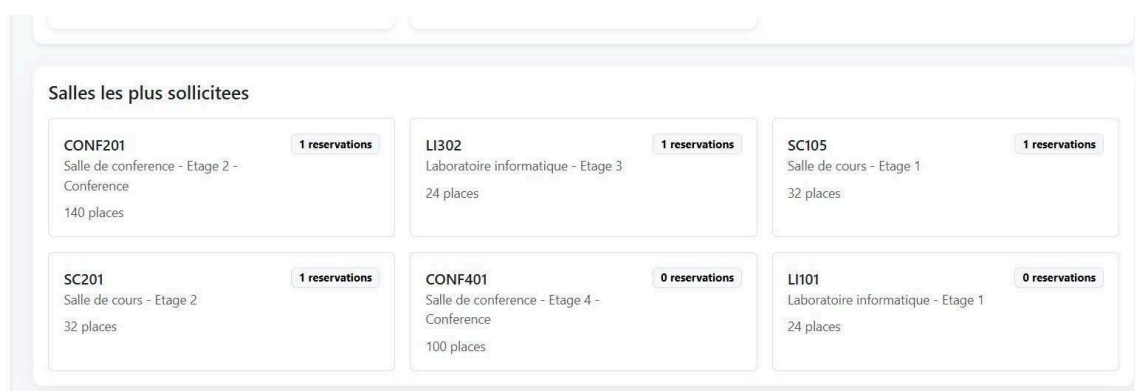


Figure 4.8 – Interface – Salles les plus sollicitées

4.4.5 Gestion des réservations (Mes demandes)

La page Mes demandes permet à chaque utilisateur de suivre le statut de ses réservations, de les modifier ou de les annuler. Les statuts sont codés par badges colorés.

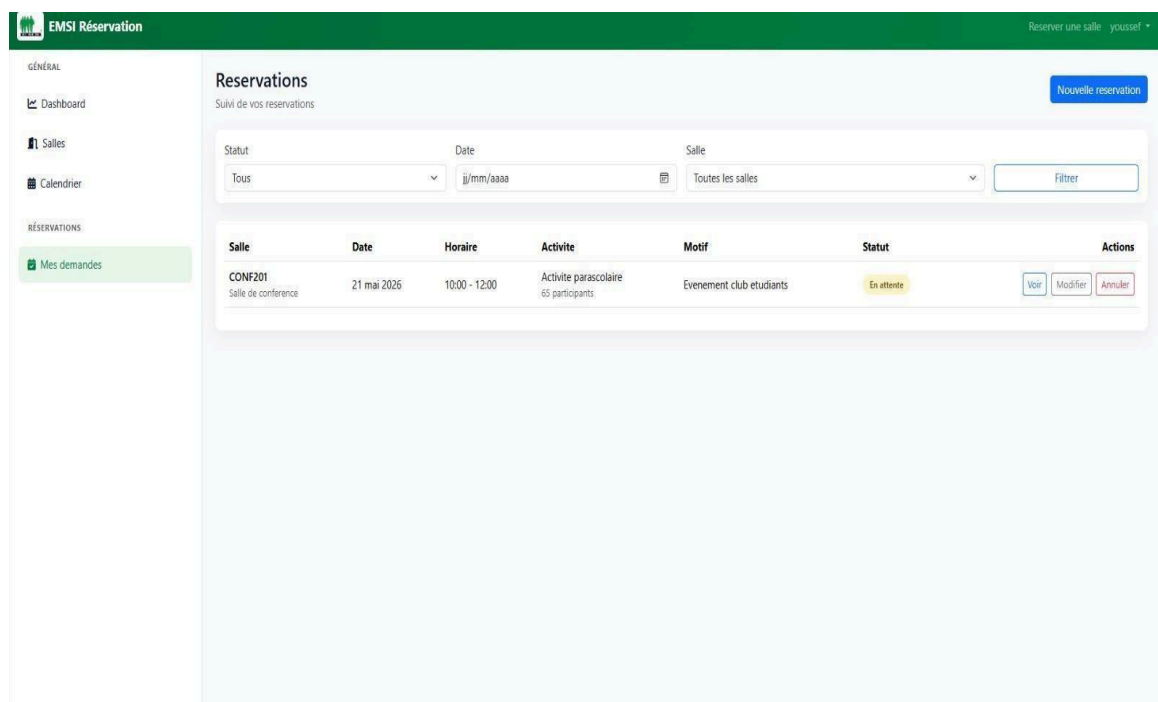


Figure 4.9 – Interface – Mes demandes de réservation

Statuts de réservation

- EN ATTENTE – En attente de validation par l'administration
- CONFIRMÉE – Réservation approuvée par l'administration
- REFUSÉE – Réservation rejetée
- ANNULÉE – Réservation annulée par l'utilisateur

4.4.6 Calendrier des réservations

Le module calendrier offre une vue hebdomadaire ou mensuelle de toutes les réservations. L'utilisateur peut filtrer par salle et par statut. Les créneaux sont codés par couleur (vert pour approuvé, jaune pour en attente).

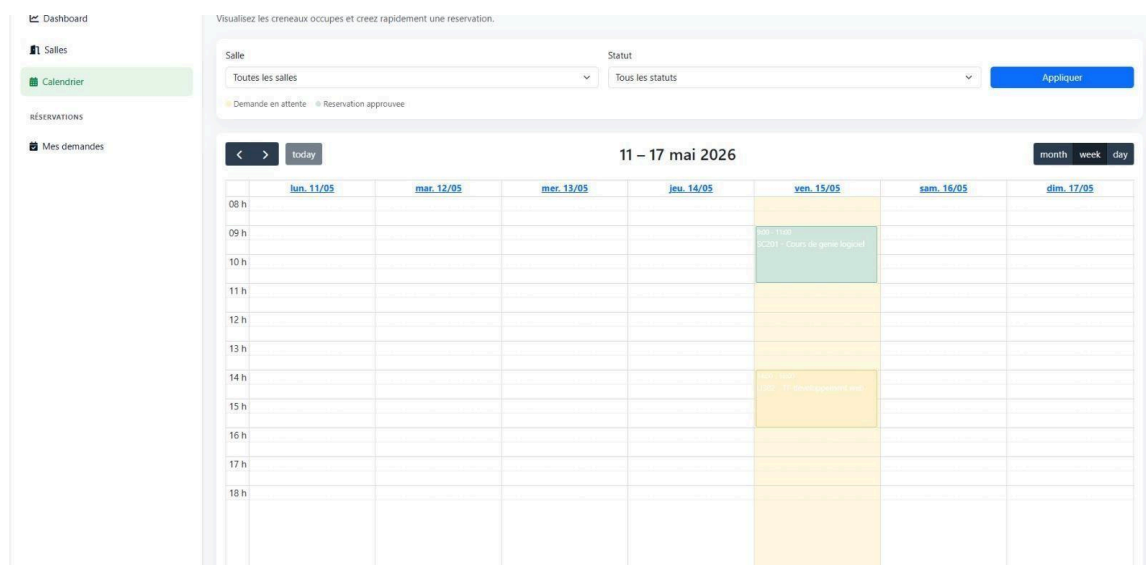


Figure 4.10 – Interface – Calendrier des réservations (vue semaine)

4.5 Interfaces admin – Frontend

4.5.1 tableau de bord(Dashboard) admin

. Cette interface responsive, développée avec Bootstrap 5, fait office de centre de pilotage visuel en regroupant plusieurs indicateurs clés mis à jour en temps réel. On y distingue trois sections majeures :

- * Les cartes métriques (KPIs) : Situées en haut, elles affichent de manière synthétique le nombre total de réservations actives (3), le flux du jour actuel (0), le volume global des salles gérées (52) et les demandes en attente de validation (2).
- * Les widgets statistiques : > * Parc EMSI : Détaille la répartition immobilière (40 salles de cours, 10 laboratoires informatiques et 2 salles de conférence).
- * Qualité du flux : Suit les performances de traitement de l'administration (demandes approuvées, refusées ou annulées).
- * Le tableau des réservations récentes : Positionné à droite, il liste les dernières activités enregistrées (ex: demandes de Youssef Alaoui ou Salma Karim), permettant un suivi rapide de l'état des créneaux (Approuvée / En attente) sans avoir à naviguer dans les sous-menus.

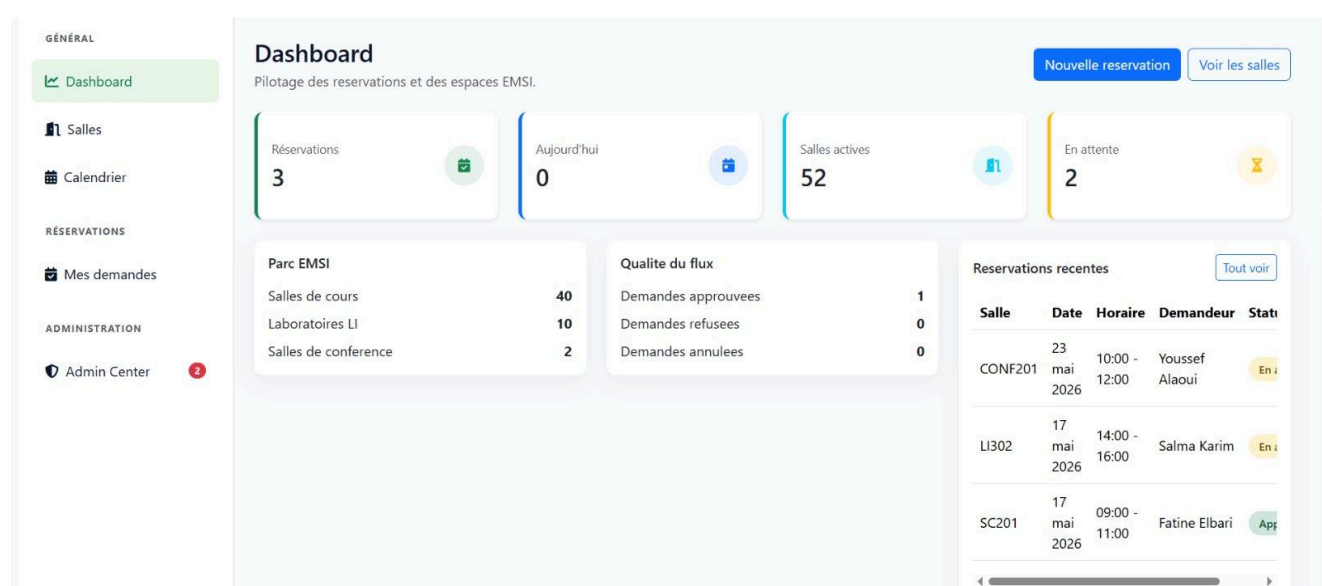


Figure 4.11 – Interface – Tableau de bord (Dashboard)

4.5.2 Interface de modération (Vue Administration)

Cette interface illustre le centre de modération mutualisé accessible par l'acteur Administration. Conçue pour simplifier le flux de travail (workflow), elle permet de traiter les demandes en attente de manière groupée ou individuelle. Les fonctionnalités clés présentées incluent :

- * Filtres de supervision : Des menus déroulants permettent de filtrer dynamiquement le flux par statut (ex: "En attente"), profil du demandeur (Enseignant/Élève), type d'activité ou étage.
- * Actions de modération : L'administrateur dispose d'un champ de texte pour saisir un message d'explication ("Message admin mutualisé") qui sera automatiquement envoyé au demandeur, accompagné de boutons d'action rapide : Approuver la sélection, Refuser la sélection ou Annuler.
- * Tableau des demandes en attente : Il expose les détails techniques de chaque créneau soumis : le nom de la salle avec son étage, le type d'activité (ex: "Travaux pratiques - TP de développement web"), la date, la plage horaire et l'indice de charge (ex: 20 participants sur 24 places disponibles pour la salle LI302).
- * Dernières décisions admin : Un historique en bas de page retrace les derniers arbitrages effectués (ex: la validation de la salle SC201 pour Fatine Elbari par le système de détection automatique).

Filtres de supervision

Statut
En attente

Profil
Tous

Activité
Toutes

Etage
Tous

Recherche
Salle, nom, objet...

Appliquer

Reinitialiser

Message admin mutualisé

Message interne ou retour envoyé au demandeur.

Actions de modération

Approuver la sélection

Refuser la sélection

En attente

Annuler la sélection

Le message saisi ici sera réutilisé pour chaque demande sélectionnée.

	Salle	Demandeur	Activité	Date	Charge	Statut	Detail
☐	LI302 Laboratoire informatique - Etage 3	Salma Karim Enseignant	Travaux pratiques TP developpement web	17 mai 2026 14:00 - 16:00	20 / 24	En attente	Voir
☐	CONF201 Salle de conference - Etage 2 - Conference	Youssef Alaoui Eleve	Activite parascolaire Evenement club etudiants	23 mai 2026 10:00 - 12:00	65 / 140	En attente	Voir

Dernieres decisions admin

Salle	Demandeur	Decision	Par	Date de traitement
SC201	Fatine Elbari	Approuvee	Systeme	16 mai 2026 15:03

Figure 4.12 – Interface de modération (Vue Administration)

4.5.3 Vue capacitaire et statistiques (Admin Center EMSI)

Cette capture d'écran dévoile l'Admin Center EMSI, le tableau de bord décisionnel avancé conçu pour l'arbitrage des priorités et la supervision des flux énergétiques et capacitaires au sein de l'établissement. L'écran est divisé en deux modules analytiques majeurs :

* Vue capacitaire par étage : Ce composant cartographie la charge sur les 5 étages du campus. L'indice de charge est calculé dynamiquement par la formule suivante :

$$\text{Indice de flux} = \text{Demandes en attente} + \text{Demandes approuvées}$$
 Il permet d'identifier visuellement et instantanément les goulots d'étranglement (ex: l'Étage 2 enregistre un flux de 2 demandes actives, tandis que l'Étage 3 enregistre 1). Demandes par profil et type d'activité : Situé sur le volet droit, ce bloc fournit des graphiques et compteurs segmentés. Il indique à l'administration l'origine sociologique des demandes (1 Enseignant, 1 Élève) ainsi que la nature pédagogique des réservations (1 TP, 1 Cours, 1 Activité parascolaire), facilitant ainsi une gestion prédictive et optimisée des espaces immobiliers

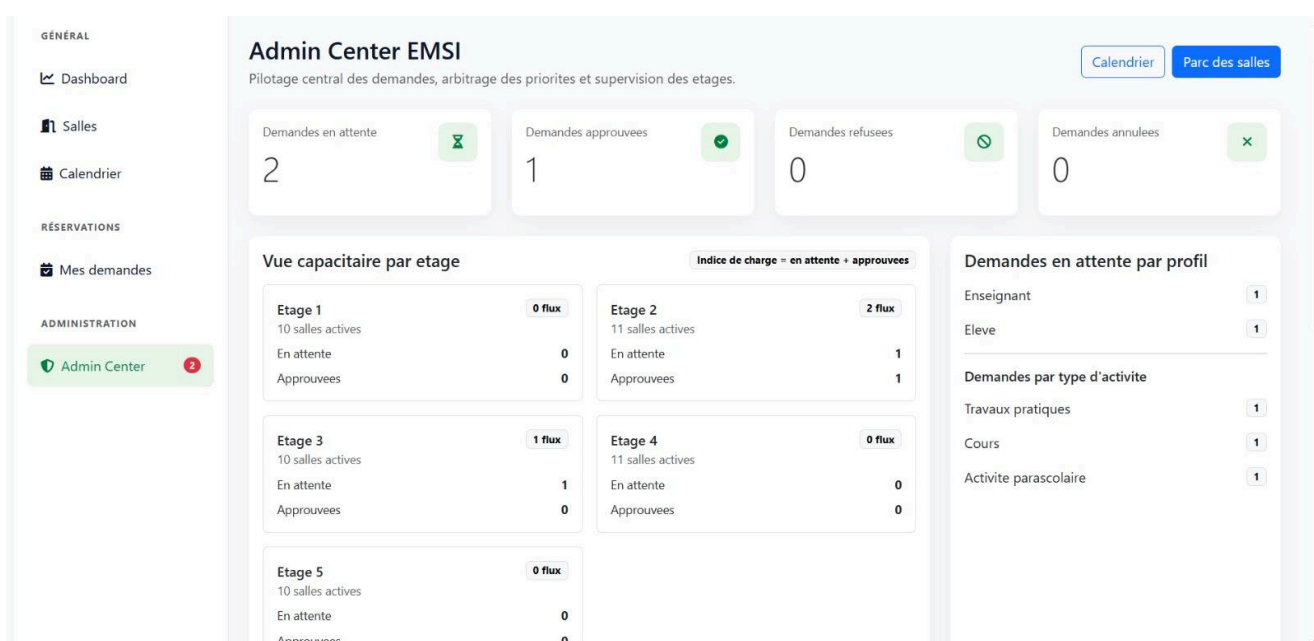


Figure 4.13 – Vue capacitaire et statistiques (Admin Center EMSI)

4.6 Détection des conflits de réservation

La logique de détection de conflits est implémentée au niveau du sérialiseur DRF. Lors de la soumission d'une nouvelle réservation, le système vérifie qu'aucune réservation confirmée ou en attente n'existe sur le même créneau pour la même salle.

4.6.1 Listing 4.1 – Modèle Booking (models.py)

```
class Booking(models.Model):

    STATUS_PENDING = 'EN_ATTENTE'
    STATUS_APPROVED = 'CONFIRMEE'
    STATUS_REJECTED = 'REFUSEE'
    STATUS_CANCELLED = 'ANNULEE'

    STATUS_CHOICES = [
        (STATUS_PENDING, 'En attente'),
        (STATUS_APPROVED, 'Approuvee'),
        (STATUS_REJECTED, 'Refusee'),
        (STATUS_CANCELLED, 'Annulee'),
    ]

    ACTIVITY_CLASS = 'COURS'
    ACTIVITY_PRACTICAL = 'TP'
    ACTIVITY_MEETING = 'REUNION'
    ACTIVITY_CLUB = 'CLUB'
    ACTIVITY_WORKSHOP = 'ATELIER'
    ACTIVITY_EVENT = 'EVENEMENT'

    ACTIVITY_TYPE_CHOICES = [
        (ACTIVITY_CLASS, 'Cours'),
        (ACTIVITY_PRACTICAL, 'Travaux pratiques'),
        (ACTIVITY_MEETING, 'Reunion'),
        (ACTIVITY_CLUB, 'Activite parascolaire'),
        (ACTIVITY_WORKSHOP, 'Atelier'),
        (ACTIVITY_EVENT, 'Evenement'),
    ]
    user = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)
    room = models.ForeignKey(Room, on_delete=models.CASCADE)
    date = models.DateField()
    start_time = models.TimeField()
    end_time = models.TimeField()
    status = models.CharField(max_length=20, choices=STATUS_CHOICES,
default=STATUS_PENDING)
    activity_type = models.CharField('Type d activite', max_length=20,
choices=ACTIVITY_TYPE_CHOICES,
default=ACTIVITY_MEETING)
    attendees_count = models.PositiveIntegerField('Nombre de participants',
default=1)
    purpose = models.CharField(max_length=255)
    admin_message = models.TextField('Message de l administration',
blank=True)
    reviewed_by = models.ForeignKey(settings.AUTH_USER_MODEL,
on_delete=models.SET_NULL, null=True, blank=True,
related_name='reviewed_bookings')
```

```

reviewed_at      = models.DateTimeField(null=True, blank=True)
created_at       = models.DateTimeField(auto_now_add=True)

objects = BookingManager()

class Meta:
    ordering = ['date', 'start_time']
    constraints = [
        models.UniqueConstraint(
            fields=['room', 'date', 'start_time'],
            name='unique_room_date_start_time',
        )
    ]
    indexes = [
        models.Index(fields=['room', 'date']),
        models.Index(fields=['user', 'date']),
    ]

```

4.6.2 Modèle Room (models.py)

```

class Room(models.Model):

    TYPE_CLASSROOM = 'CLASSROOM'
    TYPE_LAB = 'LAB'
    TYPE_CONFERENCE = 'CONFERENCE'

    ROOM_TYPE_CHOICES = [
        (TYPE_CLASSROOM, 'Salle de cours'),
        (TYPE_LAB, 'Laboratoire informatique'),
        (TYPE_CONFERENCE, 'Salle de conference'),
    ]

    buildin      = models.CharField(max_length=100, default='EMSI')
    g floor      = models.PositiveSmallIntegerField(default=1)
    code         = models.CharField(max_length=20, unique=True,
    null=True)   blank=True,
    name
    room_type    = models.CharField(max_length=100, blank=True)
    choices=ROOM_TYPE_CHOICES, default=TYPE_CLASSROOM)
    capacity     = models.PositiveIntegerField()
    location     = models.CharField(max_length=255, blank=True)
    description  = models.TextField(blank=True)
    equipment    = models.TextField('Equipements', blank=True,
        help_text='Liste des equipements disponibles (ex:
    projecteur, tableau, wifi)')
    is_active    = models.BooleanField(default=True)
    created_at   =
    models.DateTimeField(auto_now_add=True) updated_at
    = models.DateTimeField(auto_now=True)

    class Meta:
        ordering = ['floor', 'code']

    def _str(self): return
    self.code

```

4.6.3 Listing 4.2 – Logique de détection de conflits (serializers.py)

La méthode `validate()` du sérialiseur implémente une validation complète : vérification des horaires, de la date, de l'activité de la salle, de la capacité, et enfin la détection des conflits de créneaux.

```
def validate(self, attrs):
    start_time =
    attrs.get('start_time') end_time =
    attrs.get('end_time') booking_date =
    attrs.get('date')
    room = attrs.get('room')

    if start_time and end_time and start_time >= end_time:
        raise serializers.ValidationError(
            "L'heure de fin doit etre apres l'heure de debut.")

    if booking_date and booking_date < datetime.now().date():
        raise serializers.ValidationError(
            'La date ne peut pas etre dans le passe.')

    if room and not room.is_active:
        raise serializers.ValidationError(
            'Cette salle est inactive et ne peut pas etre reservee.')

    attendees_count = attrs.get('attendees_count')
    if room and attendees_count and attendees_count > room.capacity:
        raise serializers.ValidationError(
            'Le nombre de participants depasse la capacite de la salle.')

    if booking_date and room and start_time and end_time:
        conflicts = Booking.objects.filter(
            room=room,
            date=booking_date,
            status_in=[Booking.STATUS_PENDING, Booking.STATUS_APPROVED],
            start_time_lt=end_time,
            end_time_gt=start_time,
        )
        if self.instance:
            conflicts = conflicts.exclude(pk=self.instance.pk)
        if conflicts.exists():
            conflicting_bookings = [
                f"{(b.start_time)-(b.end_time)} {(b.user.username)}"
                for b in conflicts
            ]
            raise serializers.ValidationError(
                f"Conflit detecte: la salle est reservee: {'',
                '.join(conflicting_bookings)}")
        )
    return attrs
```

CHAPITRE 5

Résultats, tests et validation

5.1 Résultats obtenus

Le système SRIS développé répond à l'ensemble des objectifs fixés dans le cahier des charges. À l'issue du développement, les résultats suivants ont été obtenus :

Fonctionnalité	Statut	Remarques
Authentification par rôle	✓ Implémentée	4 rôles distincts
Gestion des salles (CRUD)	✓ Implémentée	52 salles créées
Réservation en ligne	✓ Implémentée	Formulaire web + API
Détection des conflits	✓ Implémentée	Validation côté serveur
Calendrier visuel	✓ Implémentée	Vue semaine / mois / jour
Tableau de bord admin	✓ Implémentée	Indicateurs en temps réel
API REST (DRF)	✓ Implémentée	Endpoints CRUD complets
Notifications email	⚠ Partielle	Envoi configuré, à tester
Interface responsive	✓ Implémentée	Bootstrap 5

5.2 Tests réalisés

5.2.1 Tests fonctionnels

Scénario de test	Rôle	Résultat	Observations
Connexion avec identifiants valides	Tous	✓ PASS	Redirection dashboard
Connexion avec identifiants invalides	Tous	✓ PASS	Message d'erreur affiché
Réservation sans conflit	Étudiant	✓ PASS	Statut EN_ATTENTE créé
Réservation avec conflit	Enseignant	✓ PASS	Erreur 400 retournée
Validation d'une réservation	Admin	✓ PASS	Statut CONFIRMÉE mis à jour
Annulation d'une réservation	Étudiant	✓ PASS	Statut ANNULÉE mis à jour
Filtrage des salles	Tous	✓ PASS	Filtres fonctionnels
Affichage calendrier	Tous	✓ PASS	Créneaux correctement affichés

5.2.2 Tests de performance

Opération	Temps moyen	Évaluation
Chargement du dashboard	0,8 s	★★★★ Excellent
Liste des salles (52 entrées)	0,5 s	★★★★ Excellent
Soumission d'une réservation	0,3 s	★★★★ Excellent
Chargement du calendrier	1,1 s	★★★ Bon
Requête API Booking List	0,2 s	★★★★ Excellent

5.3 Analyse critique

5.3.1 Points forts

- Architecture MVT claire et maintenable
- API REST complète exposée via DRF
- Détection automatique des conflits robuste
- Interface utilisateur intuitive et responsive
- Gestion fine des rôles et des permissions (RBAC)

5.3.2 Limites et axes d'amélioration

- SQLite convient pour le développement mais présente des limites en accès concurrent élevé (migration vers PostgreSQL recommandée)
- Les notifications email nécessitent une configuration SMTP complète
- L'application n'est pas encore déployée en production (serveur web externe)
- Absence d'application mobile native (envisageable via l'API existante)

Conclusion et perspectives

Synthèse

Ce projet nous a permis de concevoir et développer le Système de Réservation Intelligente des Salles (SRIS), une application web complète répondant aux besoins réels de l'EMSI Casablanca en matière de gestion des espaces universitaires.

Du cahier des charges initial jusqu'à la mise en œuvre technique, nous avons appliqué les méthodologies apprises au cours de notre formation en ingénierie informatique : modélisation UML, développement web avec Django, API REST, gestion de base de données et bonnes pratiques de sécurité.

Bilan

Le système développé offre une couverture fonctionnelle complète : gestion des utilisateurs par rôles, consultation des salles, réservation en ligne, détection automatique des conflits, calendrier visuel et tableau de bord d'administration. L'ensemble des objectifs principaux du cahier des charges a été atteint.

Perspectives d'évolution

- Déploiement en production sur un serveur cloud (Heroku, PythonAnywhere, VPS)
- Migration vers PostgreSQL pour un accès concurrent plus performant
- Application mobile (React Native ou Flutter) exploitant l'API REST existante
- Module de rapports avancés avec export PDF/Excel
- Intégration d'un système de récurrence pour les réservations hebdomadaires
- Authentification SSO via les comptes institutionnels EMSI

Ce projet constitue une base solide et évolutive. Nous espérons qu'il pourra être adopté et enrichi pour répondre aux besoins croissants de l'institution.

Annexe A – Cahier des charges complet

Le présent cahier des charges définit les bases du Système de Réservation Intelligente des Salles (SRIS),

Contraintes techniques

- Python 3.x comme langage principal
- Django 4.x (architecture MVT)
- Django REST Framework pour l'API
- SQLite comme base de données (choix définitif du projet)
- HTML5, CSS3, JavaScript ES6+, Bootstrap 5
- Authentification via Django Authentication System
- Protection CSRF sur tous les formulaires
- Gestion des permissions par rôle (RBAC)

Contraintes de sécurité

- Authentification obligatoire pour accéder au système
- Protection contre les injections SQL via l'ORM Django
- Tokens CSRF sur tous les formulaires
- Validation des données côté serveur